Icône Ajouter Ajouter une couverture

4. Nextcloud

Certificat SSL :

Ajout de l'entrée DNS pour le sous domaine cloud :

cloud	A	51.91.127.143	Par défaut
cloud	AAAA	2001:41d0:404:200::798e	Par défaut

Déclaration du domaine dans mailcow.conf :

Shell

```
sudo nano /opt/mailcow-dockerized/mailcow.conf

ADDITIONAL_SAN=cloud.privatiq.fr
```

Redémarrage du conteneur ACME :

Shell

```
sudo docker compose restart acme-mailcow
```

Mailcow détecte automatiquement le nouveau domaine dans `ADDITIONAL_SAN`, vérifie que le DNS répond, et émet le certificat Let's Encrypt couvrant tous les domaines.

Installation du Nextcloud :

Création du docker compose.yml :

Shell

```
cd /opt/nextcloud
```

Lancer le conteneur :

Shell

```
services:
  nextcloud-app:
    image: nextcloud:latest
    container_name: nextcloud-app
    restart: unless-stopped
    ports:
      - "172.17.0.1:8080:80"
    volumes:
      - ./nextcloud_data:/var/www/html
    environment:
      - NEXTCLOUD_TRUSTED_DOMAINS=cloud.privatiq.fr
    depends_on:
      - nextcloud-db

  nextcloud-db:
    image: mariadb:10.11
    container_name: nextcloud-db
```

```
    restart: unless-stopped
    environment:
      - MYSQL_ROOT_PASSWORD=changeme
      - MYSQL_DATABASE=nextcloud
      - MYSQL_USER=nextcloud
      - MYSQL_PASSWORD=changeme
    volumes:
      - ./nextcloud_db:/var/lib/mysql
```

Shell

```
sudo docker compose up -d
```

Injection de la config nginx dans Mailcow

Création du fichier `/opt/mailcow-dockerized/data/conf/nginx/nextcloud.conf` :

Shell

```
server {
    listen 80;
    listen [::]:80;
    server_name cloud.privatiq.fr;

    # Exception ACME avant la redirection HTTPS
    location ^~ /.well-known/acme-challenge/ {
        allow all;
        default_type "text/plain";
        root /web;
    }

    location / {
        return 301 https://$host$request_uri;
    }
}


server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name cloud.privatiq.fr;

    # Mailcow gère les certs Let's Encrypt automatiquement
    ssl_certificate /etc/ssl/mail/cert.pem;
    ssl_certificate_key /etc/ssl/mail/key.pem;

    # Headers requis par Nextcloud
    add_header Strict-Transport-Security "max-age=15768000; includeSubDomains; preload"
always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options SAMEORIGIN always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy no-referrer always;

    client_max_body_size 10G;
    proxy_read_timeout 600s;
    proxy_connect_timeout 600s;

    location / {
        proxy_pass http://host.docker.internal:8080;
        # ou 172.17.0.1:8080 selon ta config Docker
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Optimisation CalDAV/CardDAV
    location = /.well-known/carddav {
        return 301 $scheme://$host/remote.php/dav;
```

```
    }
    location = /.well-known/caldav {
        return 301 $scheme://$host/remote.php/dav;
    }
}
```

Problème rencontré :

Nextcloud affiche Mailcow — Problème IPv6 dans nginx

Symptôme

En accédant à `https://cloud.privatiq.fr`, la page de Mailcow s'affiche à la place de Nextcloud, malgré une config nginx `nextcloud.conf` correcte.

Cause

Le VPS OVH répond en **IPv6** (`2001:41d0:404:200::798e`). Or le fichier `nextcloud.conf` n'écoutait que sur IPv4. Nginx recevait la requête sur `[::]:443` et, ne trouvant pas de vhost IPv6 correspondant à `cloud.privatiq.fr`, tombait sur le vhost par défaut de Mailcow.

Diagnostic

Shell

```
# Vérifier que le serveur répond en IPv6
curl -v https://cloud.privatiq.fr 2>&1 | grep "Connected"
# → Connected to cloud.privatiq.fr (2001:41d0:404:200::798e)

```

Solution

Ajouter les directives `listen [::]:80` et `listen [::]:443 ssl` dans `/opt/mailcow-dockerized/data/conf/nginx/nextcloud.conf` :

Nginx Config

```
server {
    listen 80;
    listen [::]:80;
    server_name cloud.privatiq.fr;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    http2 on;
    server_name cloud.privatiq.fr;

    ssl_certificate /etc/ssl/mail/cert.pem;
    ssl_certificate_key /etc/ssl/mail/key.pem;

    add_header Strict-Transport-Security "max-age=15768000; includeSubDomains; preload" a
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options SAMEORIGIN always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy no-referrer always;

    client_max_body_size 10G;
```

```
    proxy_read_timeout 600s;
    proxy_connect_timeout 600s;

    location / {
        proxy_pass http://172.17.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location = /.well-known/carddav {
        return 301 $scheme://$host/remote.php/dav;
    }
    location = /.well-known/caldav {
        return 301 $scheme://$host/remote.php/dav;
    }
}

```

Puis recharger nginx :

Shell

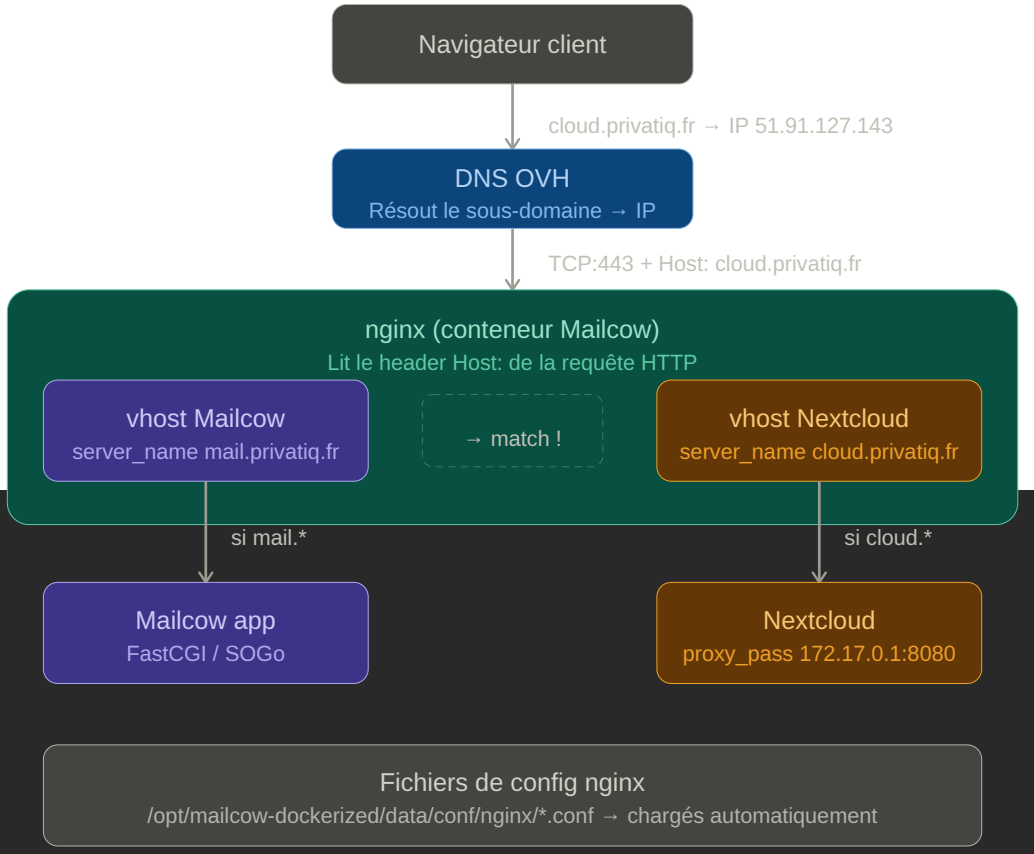
```
sudo docker exec mailcowdockerized-nginx-mailcow-1 nginx -s reload

```

À retenir

Sur tout VPS OVH avec une adresse IPv6, **toujours déclarer les deux directives listen** (IPv4 et IPv6) dans les vhosts nginx custom, sinon les requêtes IPv6 tombent sur le vhost par défaut.

Principe du Virtual Hosting.



Concrètement pour ajouter un nouveau service :

1. DNS OVH → ajouter un enregistrement A (et AAAA) qui pointe le sous-domaine vers ton IP
2. Fichier nginx → créer `/opt/mailcow-dockerized/data/conf/nginx/monservice.conf` avec le bon `server_name` et le `proxy_pass` vers le port Docker interne
3. Docker → exposer le service sur `172.17.0.1:PORT` (accessible depuis le conteneur nginx)
4. Nextcloud → ajouter le domaine dans `NEXTCLOUD_TRUSTED_DOMAINS` si c'est Nextcloud, sinon l'équivalent dans l'app concernée
5. Certificat → ajouter le sous-domaine dans `ADDITIONAL_SAN` de `mailcow.conf` et redémarrer acme

C'est tout — nginx fait le tri tout seul grâce au `Host:` header.